Kutty Banerjee

kutty.banerjee@gmail.com (925)-209-7204

Summary

Experienced in System Software, Computer Architecture, Parallel Computing, Compilers, SW/HW boundaries [Hardware Verification Infrastructures]. Seeking a Senior Architect [Management Position].

Experience

Apple : iOS GPU Software Engineer [Aug 2013 – Present]

- Added a "Multi Linear Regresison Model" for Predicting GPU Power consumption at runtime based on GPU utilization counters.
- Augmented Firmware with a Credit based Policy for Fairness in GPU Work Scheduling.
- Added GPU Compiler and Driver support to implement efficient vertex clipping for user planes (OpenGL)
- Added Data Flow logic in Driver to automatically eliminate CPU/GPU Cache Flushes for OpenCL kernels that do not exhibit RAW/WAW dependencies.

Intel : GPU Platform Architect [May 2013 – Aug 2013]

- Drive Low Power design of Compute features on next-gen Intel GPUs.

NVIDIA : Software Architect [May 2005 – May 2013]

Project Denver (NVIDIA's ARM64 CPU)

- Did CPU Perf Analysis of Graphics, Web and JVM workloads and coordinated with HW and various SW teams.
- Identified Instruction Scheduling Stalls / Missing Speculative Optimizations/Excessive Rollback of Instructions /Cache bottlenecks.
- Implemented a Memory Profiler and augmented CPU/GPU Functional Simulators to use it. Cost an additional 15-20% overhead at runtime. Captured Physical/Virtual Pages accessed from various threads, cores along with Page attributes. Data mined this using a Database offline to Graph Page Access Patterns, Page-Migration Patterns for CPU/GPU workloads.
- Denver uses a Hypervisor to Trace JIT compile ARM to native VLIW after Trace Profiling. I maintained the Trace Profiler and Feedback codebase. As part of that role implemented:
  - Branch Assert instruction to remove cold branches and improve Instruction Scheduling
  - Lazy Branch Assert to avoid recompiling traces for occasionally taken cold branches
  - Branch Reprofiling to detect dynamic changes in branch weights of a running program and trigger recompilation
  - Elide Fence instructions for speculative instructions and provide a rollback if during commit another core accessed the speculative lines. Helps JVM workloads
- Added Large Page Support in Tegra Linux kernel using a BSD-like Lazy Page Coalescing approach.
- Added "Thread Local Heap" support in Tegra kernel to elide TLB invalidate IPIs and cache snoop for private memory. Helps Browser workloads.
- Worked with various Driver teams to enforce code was safe for ARM Weak Consistency Rules and AXI Bus ordering rules.

Project Orion (CUDA) – Cache Coherent CPU/GPU on a single SoC

- Was responsible for delivering UVM (Unified Virtual Memory) across CPU and GPU on a Custom NVIDIA SoC with an internal CPU GPU and Cache-Coherent Interconnect for the CUDA HPC Market.
- My role was to drive adding features in GPU, SoC and the Driver stack to make this happen. This included adding support for GPU Page Fault handling, GPU TLB redesign, Page Migration logic, interrupt coalescing and low latency interrupt delivery.
- Designed ARM64 Linux kernel changes to make this possible. Code was written by another team member.
- Implemented GPU Functional and Perf Simulator changes to test Performance impact of False-sharing and Page Ping Pong.

- Project was later cancelled but the above worked move to a different project

Parallel Computing Framework:

- During Sabbatical, implemented "**Khronos = A SW implementation of an Out-Of-Order processor**"
- It's a Stack Based Virtual Machine with its own Instruction Set. Instructions are very similar to OpenCL kernels and Operands are Tiled Regions of Memory.
- Instruction Parallelism is obtained by Data Flow analysis of Instructions forming an Anti-Dependency Graph which is then parsed by separate CPU Threads in Parallel to obtain Parallel execution for independent instructions.
- Supports Branches via Predication and first class Instructions for loops. Also has a special instruction for Parallel Reduction.
- Implemented a Task Parallel Library with efficient locking characteristics and various concurrency primitives as building blocks.
- Added a Data-Aware Scheduler to factor in memory traffic induced by migrating work to GPU.
- Implemented various Atomics/Fence free/Transactional algorithms. Implemented Biased locking to remove CAS and fence instructions from uncontested locks.

GPU Test Infrastructure Lead

- Lead a Cross-Team effort to unify verification efforts of various Hardware and Software/Driver teams on the MDIAD-MODS Platform.
- MDIAG is NVIDIA's Verification Platform where Kernel and OpenGL drivers are compiled as userland libraries and loaded on a micro-Linux installation. Also includes a Trace-Player to playback realworld CPU/GPU app traces. Works with Silicon and Simulators and targets Directed Deterministic verification of the Hardware.
- Added "Probe Based Verification" to Policy-Manager(PM) PM is the layer responsible for inducing faults/events and boundary conditions that allow testing hard-to-hit conditions deterministically.
- Probes are hooks in Drivers which can be programmed at runtime using a script to wait for asynchronous conditions to occur. Allows an external script to force certain hard-race conditions to occur and ensure Driver works correctly.
- Coordinated with Hardware and Software teams working on PCI-E, Display, Cache/TLB, Clocks, Core-Graphics and Video to beef up verification using the above Framework. Role was to work as an Evangelist of the MDIAG framework and beef up coverage.
- Implemented SM2, NVIDIA's Automated Test Infrastructure. This was my side project that is used today by 4 Driver/Firmware teams.
- SM2 Provides a Schema for Test Description, Test Dependencies, Policy Manager Files if any, and Test Installation Prerequisites in a INI file. This is then parsed, dependencies fetched, tests executed and results stored in a central database. Tests can be hierarchically combined, excluded, augmented and launched remotely. Implemented using Java and C

Fermi Generation GPU (DX11 compatible) Kernel Lead

- As the Kernel Driver lead, **lead a team of 25 engineers in the Resource Manager Team** and oversaw development across Simulation, RTL, Emulation and Bringup.
- Role included Task Assignment, Initial Triage, Interaction with Hardware and other Software teams, mentoring new Engineers, Interns, Code Review. Responsible for overall progress of the Kernel team for Fermi.
- Was the first engineer to start work on the Kernel/Firmware for Fermi. Wrote and helped large parts of the kernel code including Thread Scheduling, Interrupt Handling, Memory Management, Page Table Setup, Context Switch Support using Firmware, Cache mgmt., TLB invalidate batching.
- Augmented Functional Simulator's Cache and Memory modeling pipeline.

Fermi Command Buffer Scheduling Lead

- Lead the Fermi Command Buffer Scheduling unit in the Driver. Fermi added Hardware support for DX11 Per Node Command Buffers. Replaced the old Timeslice Based Scheduler with Per Node Queues.
- Added support to hold off Buffer submits (throttle) to enable Engine Power Gating
- Did plenty of code refactoring wrote lots of tests. Code had around 90% branch code coverage!!

Resource Manager Graphics Driver

- Worked on various parts of the kernel driver, fixed bugs, did triaging.
- Implemented a Non Stalling Lock Free Interrupt Tree for routing DXG kernel notifiers without delay

Education

MS Computer Science, Worcester Polytechnic Institute, MA [2003 Aug – 2005 May]

- Thesis: Remote Execution of 3D Graphics for Mobile Systems. Implemented a Memory and Power Model for Graphics computations on Battery constrained devices.
- Recipient of WPI Institute Fellowship Award for outstanding research work.

BE Computer Science, University of Mumbai [1999 July – 2003 May]

Publications

http://www.pseudoparallel.com/publications